GATE
CS & DA

Artificial Intelligence

Un-informed Search

Lecture No. 4

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic — BFS
- Topic — DFS
- Topic
- Topic
- Topic

Topics to be Covered

Topic — DFS has 2 variant ⇒ LDFS

Topic — IDDFS

Topic — UCS.

Topic

Topic

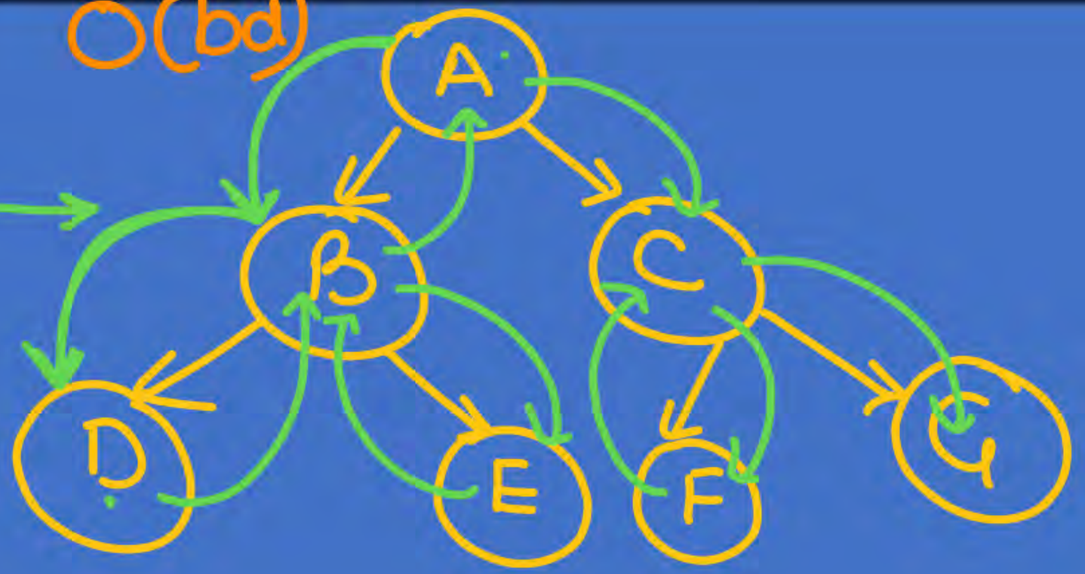
Revision

BFS vs DFS

$O(b^d)$

$O(b^d)$

$O(bd)$



* DFS ⇒ It may stuck into any ∞ length branch. and may not be able to reach result

⇒

2 Solutions

1) DLS
2) IDDFS.

Depth Limited Search (DLS)

- Depth Limited Search is a modified version of DFS that imposes a limit on the depth of the search. This means that the algorithm will only explore nodes up to a certain depth, effectively preventing it from going down excessively deep paths that are unlikely to lead to the goal. By setting a maximum depth limit, DLS aims to improve efficiency and ensure more manageable search times.

→ It may give no result if the goal state is below the mentioned depth.

Depth Limited Search (DLS)

DFS algo ⇒

- How Depth Limited Search Works
- Initialization: Begin at the root node with a specified depth limit.
- Exploration: Traverse the tree or graph, exploring each node's children.
- Depth Check: If the current depth exceeds the set limit, stop exploring that path and backtrack.
- Goal Check: If the goal node is found within the depth limit, the search is successful.
- Backtracking: If the search reaches the depth limit or a leaf node without finding the goal, backtrack and explore other branches.

Depth Limited Search (DLS)

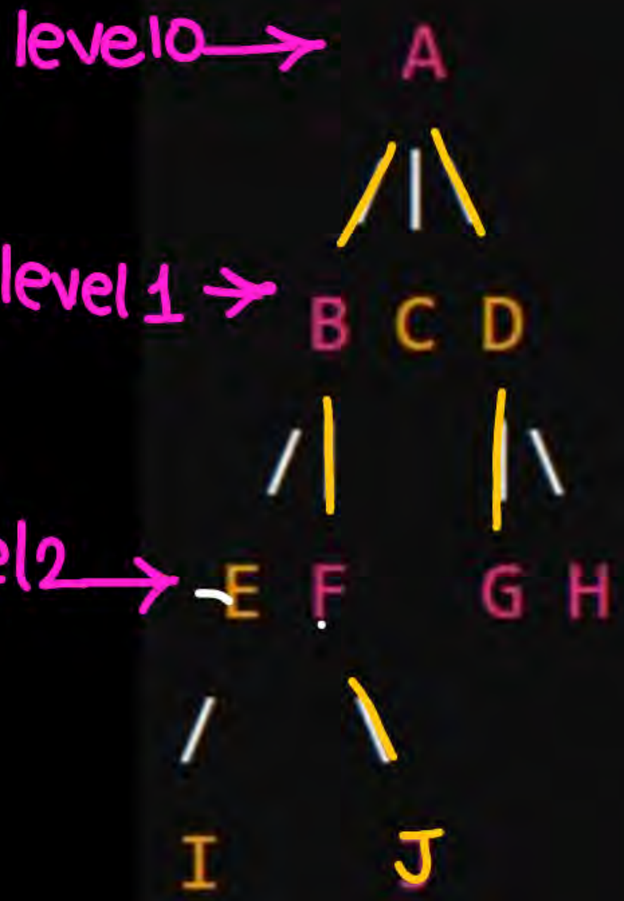
DLS DFS

Let's define our goal states as nodes G and J.
Apply DFS and DLS with depth limit 2.

- alphabetical order

DFS ⇒ ~~A~~ D C ~~B~~ ~~F~~ ~~E~~ ~~I~~ J

- DLS ⇒ ~~A~~ ~~D~~ ~~C~~ ~~B~~ ~~F~~ ~~E~~ H ~~G~~



d: depth of tree
b: branching factor.

advantage

- Will not stuck into branch with ∞ nodes.

⇒ less search time if the goal state is within given depth.

Time and space Complexity

- TC ⇒ $O(b^L)$

  L: depth limit given for DLS

- SC ⇒ $O(b \times L)$

disadvantage

- will not give result if the goal node is below specified depth.

Depth Limited Search (DLS)

- **Advantages of Depth Limited Search**
- Depth limited search is better than DFS and requires less time and memory space.

- To combat the disadvantages of DFS, we add a limit to the depth, and our search strategy performs recursively down the search tree.
- **Disadvantages of Depth Limited Search**
- The depth limit is compulsory for this algorithm to execute.
- The goal node may not exist in the depth limit set earlier, which will push the user to iterate further adding execution time.
- The goal node will not be found if it does not exist in the desired limit.

(l = limit, d: depth of goal)

Depth Limited Search (DLS)

- Performance Measures

algo Completeness doubtful.

- Completeness: The DLS is a complete algorithm in general except the case when the goal node is the shallowest node, and it is beyond the depth limit, i.e. l < d, and in this case, we never reach the goal node.
- Optimality: The DLS is a non-optimal algorithm since the depth that is chosen can be greater than d (l>d). Thus DLS is not optimal if l > d
- Time complexity is expressed as: It is similar to the DFS, i.e. $O(B^L)$, where L is the set depth limit.
- Space Complexity is expressed as: It is similar to DFS. O(BL), where L is specified depth limit.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- There are two common ways to traverse a graph, BFS and DFS. Considering a Tree (or Graph) of huge height and width, both BFS and DFS are not very efficient due to following reasons.

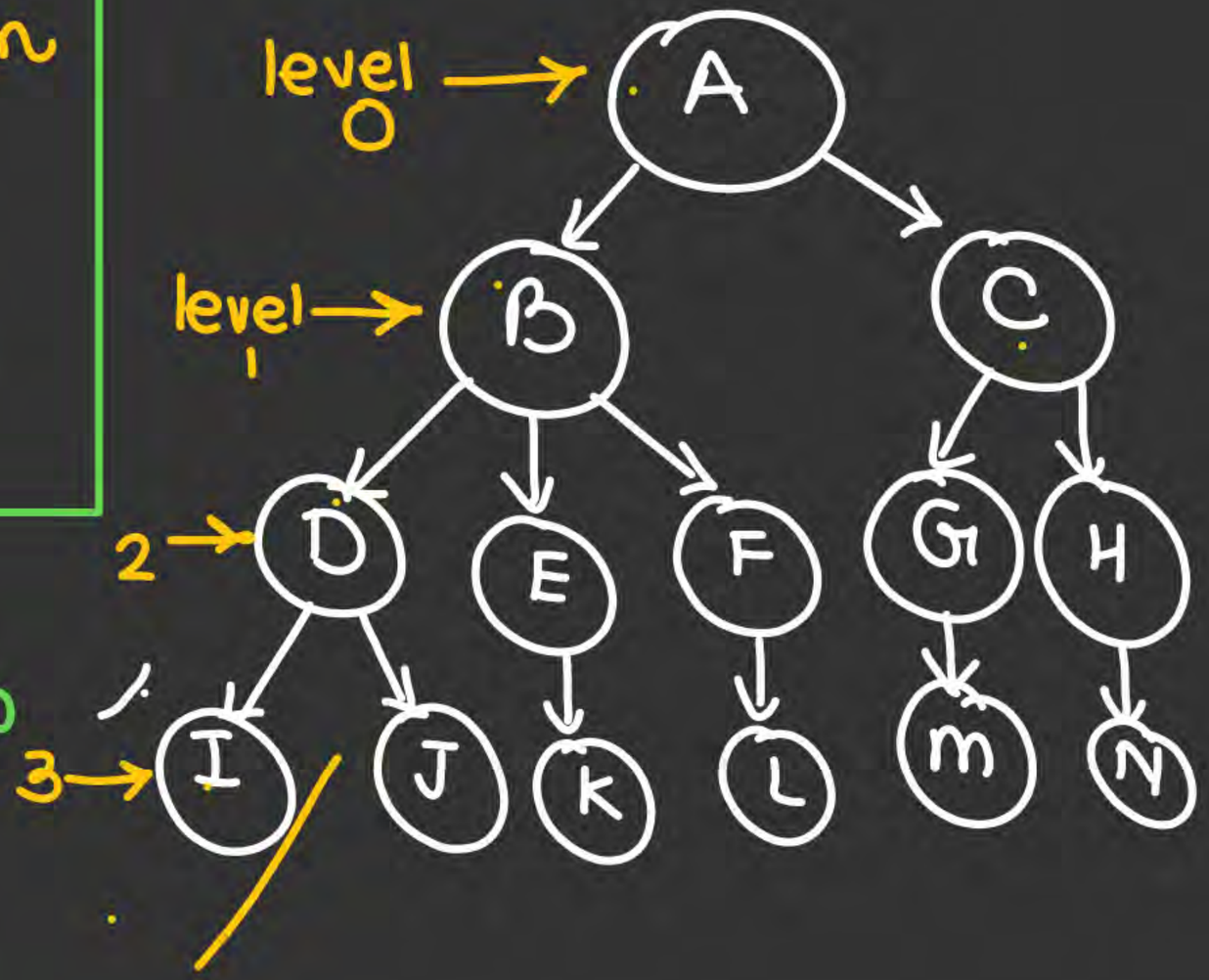
(Optimal But Space)

Spaceless, not optimal Sol.

- DFS first traverses nodes going through one adjacent of root, then next adjacent. The problem with this approach is, if there is a node close to root, but not in first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find shortest path to a node (in terms of number of edges).
- BFS goes level by level, but requires more space.

IDDFS — better than
DLS
DFS
BFS

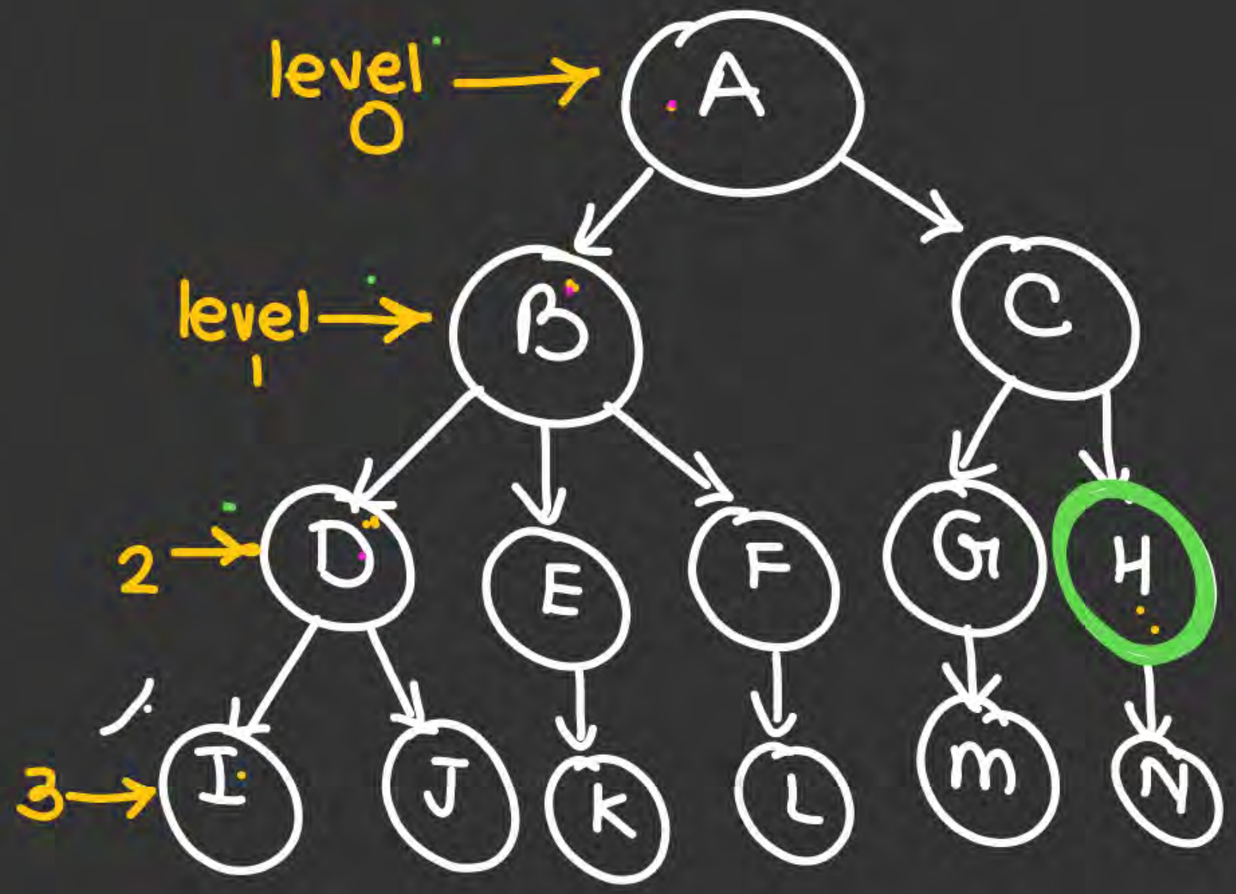
* No depth is predefine
* we start with depth limit=0
* we start with level 0.
* goal node reached no
* inc the depth limit by 1
* apply DFS



goal node (alpha)

I, H,

- BFS H
- DFS I
- IDDFS H



IDDFS

↓

Lagaya DFS

Result BFS

→ less space Complexity

→ advantage Optimum Result.

(i) Completeness ✓

(ii) optimal result

(iii) less space Complexity

One goal I

ABDI

DFS ⇒ ~~A~~ C ~~B~~ F E ~~D~~ J ~~I~~

BFS ⇒ ABC DEFGH I

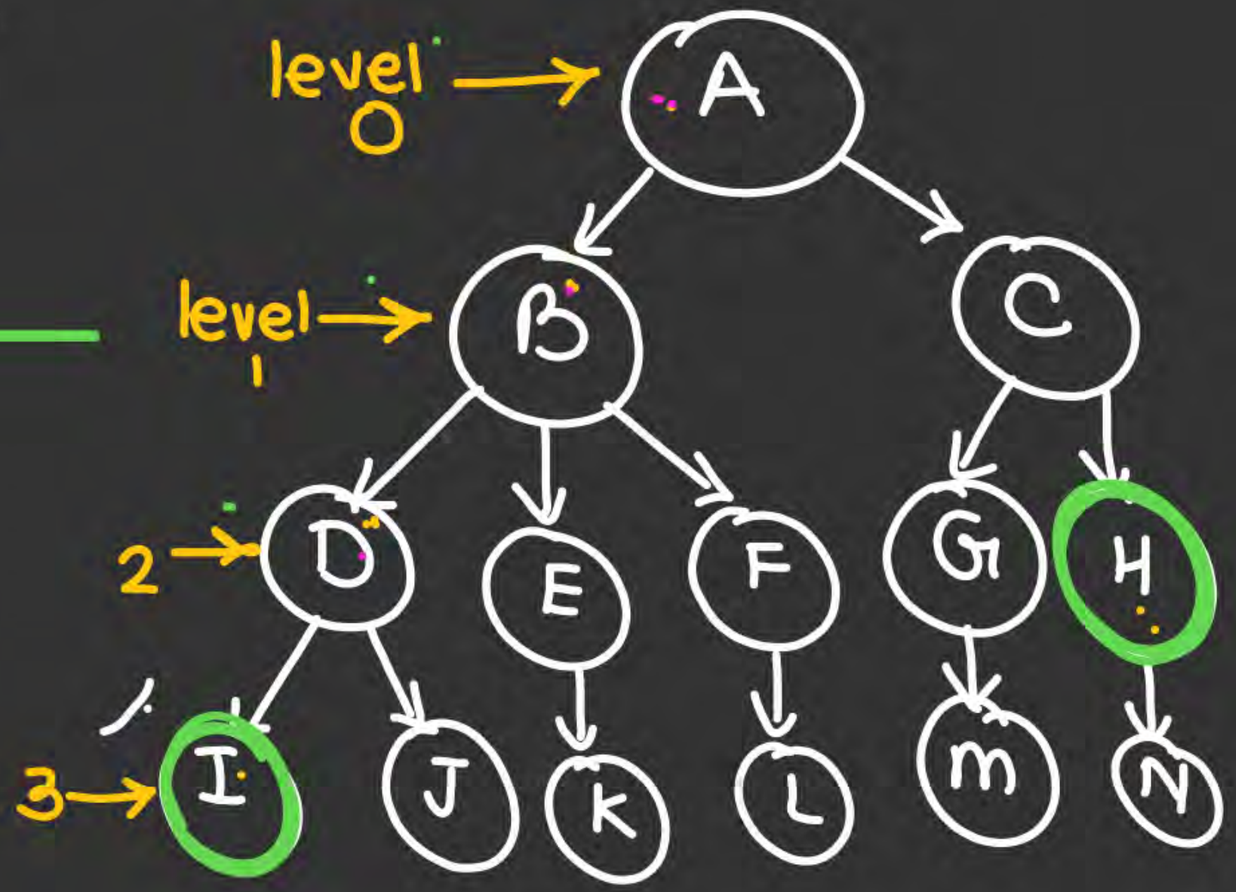
IDDFS

It 1. depth 0 A

It 2. dept 1 ABC

It 3. depth 2 ABDEFCGH

It 4. depth 3 ABDI



- b: branching factor
- G: depth of goal node

Time and Space Complexity ⇒ Time Complexity $= O(b^G)$

Space " ⇒ $O(b \times G)$.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- Key Concepts

- Depth-First Search (DFS): DFS explores as far as possible along each branch before backtracking. It uses a stack data structure (either implicitly through recursion or explicitly).
- Breadth-First Search (BFS): BFS explores all nodes at the present depth level before moving on to nodes at the next depth level. It uses a queue data structure.
- Iterative Deepening: IDDFS runs DFS repeatedly, increasing the depth limit each time until a solution is found. In the first iteration, it searches to depth 0, in the second to depth 1, and so on.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

- Iterative Deepening Depth-First Search (IDDFS) combines the depth-first search's space efficiency with the breadth-first search's completeness. It repeatedly performs depth-limited searches with increasing depth limits until the goal is found.

⇒ IDDFS use DFS and get BFS Result.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

Time Complexity:

Let's suppose b is the branching factor and depth is d then the worst-case time complexity is **$O(b^d)$**.

Space Complexity:

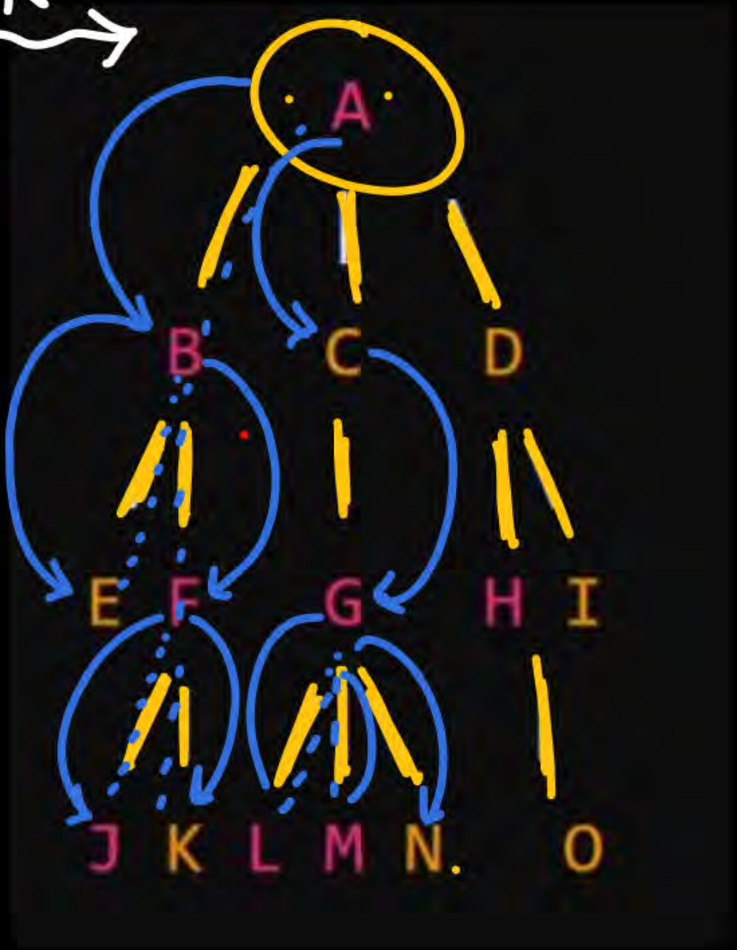
The space complexity of IDDFS will be **O(bd)**.

Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

ABEFJKCGLMN

We will search for node N.

- Path Follow by DFS (Search path, alpha) ⇒



Iterative Deepening Search (IDS) or Iterative Deepening Depth First Search (IDDFS)

Iterative way of Applying DFS ⇒

We will search for node N.

- Path Follow by IDDFS (Search path, alpha) ⇒

It.1 : depth 0 : A

It.2 : depth 1 : ~~ABCD~~ ABCD ✓

It.3. depth 2 : ~~ABCDEFGH~~

It.4. depth 3 : ~~ABCDEFGHIJKLMNO~~

ABEFJKCGLMN ✓

ABEFCGDHI

IDDFS → disadvantage

- Repeatition of DFS make the algo Complicated

Uniform Cost search

Uniform Cost Search (UCS) is a search algorithm used to find the least-cost path from a start node to a goal node in a graph. UCS is a variant of Dijkstra's algorithm and is used in situations where the path costs vary, making it more suitable for weighted graphs.

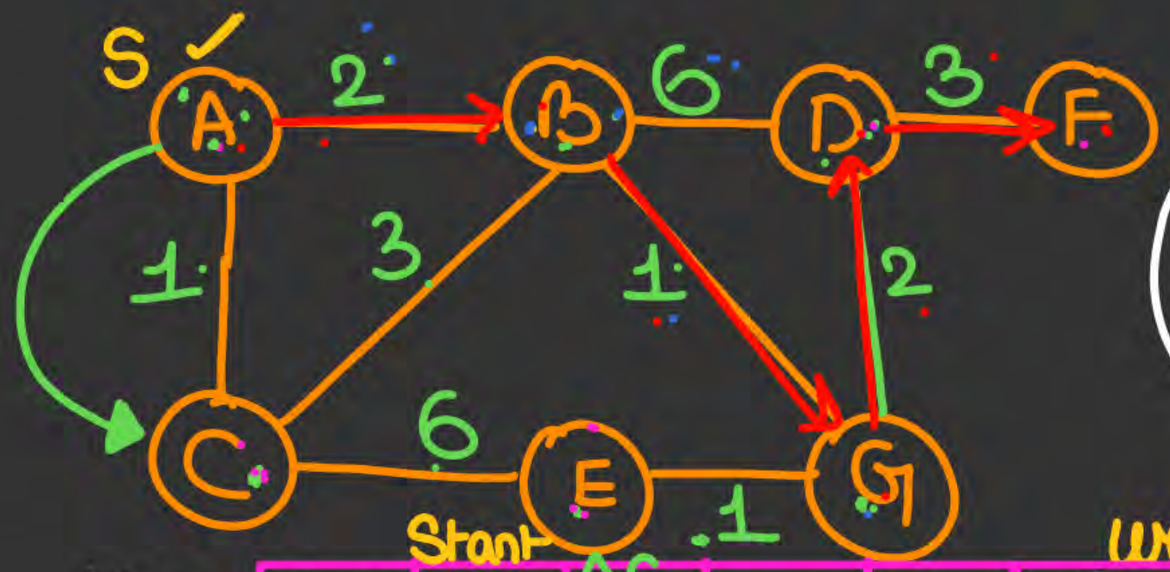
Uniform Cost search

Key Concepts

- Priority Queue: UCS uses a priority queue to explore the least-cost path first. Nodes are expanded in order of their cumulative cost from the start node.
- Cost: Each edge in the graph has a cost associated with it. UCS aims to find the path with the minimum cumulative cost.
- Expansion: The algorithm expands the node with the smallest cumulative cost, ensuring that the cheapest path is always chosen.

```
(A)---1---(B)---4---(D)---1---(F)
 |         /  \         |
 2        3    5        2
 |       /      \       |
(C)---6---(E)---1---(G)
```

Your task is to find the least cost path from city **A** to city **G** using Uniform Cost Search (UCS).



UCS → used on weighted graph

we want to move A → F

UCS

	Start	A	C	E	D
A	0	0	0	0	0
B	2	2	2	2	2
C	1	1	1	1	1
D	∞	∞	8	5	5
E	∞	7	7	4	4
F	∞	∞	∞	∞	8
G	∞	∞	3	3	3

```
        A
    /   |   \
   2    1     4
  /     |      \
 B-----C-------D
 |     |       / \
 7     3      1   5
 |     |     /     \
 E-----F---G-------H
  \    |   |       /
   8   6   2      3
    \  |  /      /
      I--------J
          9
```

Time Complexity:

Let C* **is Cost of the optimal solution**, and **ε** is each step to get closer to the goal node. Then the number of steps is = C*/ε+1. Here we have taken +1, as we start from state 0 and end to C*/ε.

Hence, the worst-case time complexity of Uniform-cost search is $\mathbf{O(b^{1 + [C^*/\varepsilon]})}$/.

Space Complexity:

The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $\mathbf{O(b^{1 + [C^*/\varepsilon]})}$.

Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

Uniform Cost search

- Algorithm Steps
- Initialization: Initialize the priority queue with the start node, assigning it a path cost of 0. Set the cost of reaching the start node to 0.
- Expand Nodes:
- While the priority queue is not empty, remove the node with the lowest path cost from the queue.
- If the removed node is the goal node, return the path and the total cost.
- For the removed node, consider all its neighbors and calculate their path costs.
- If a neighbour has not been explored or a cheaper path to it is found, update its cost and add it to the priority queue.

Uniform Cost search

- Applications
- Pathfinding in artificial intelligence and robotics.
- Network routing protocols.
- Cost optimization problems in logistics and transportation.

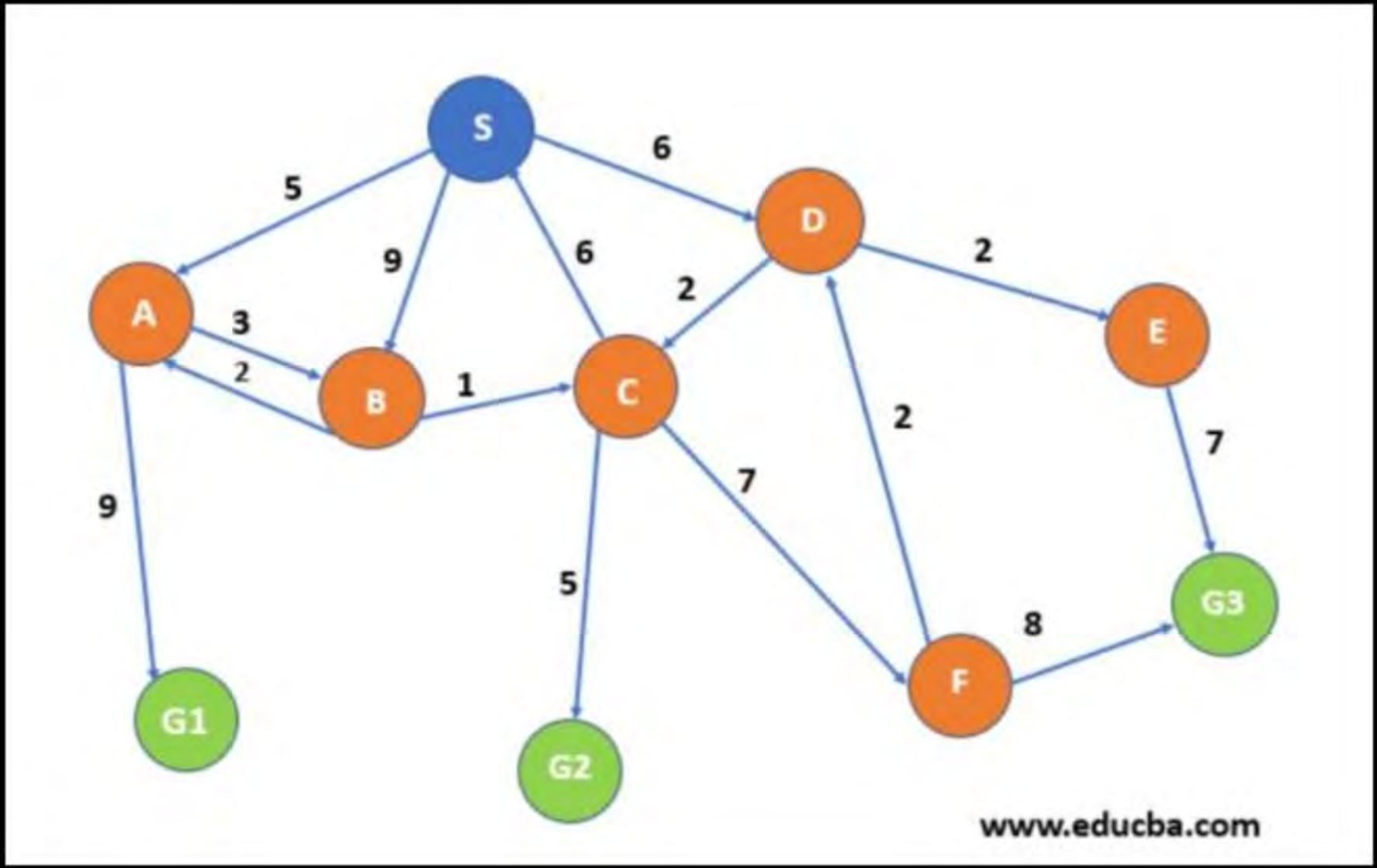
Uniform Cost search

Limitations

- UCS can be less efficient with large graphs due to its memory and time requirements.
- The algorithm assumes non-negative edge costs; negative costs can lead to infinite loops or incorrect results.

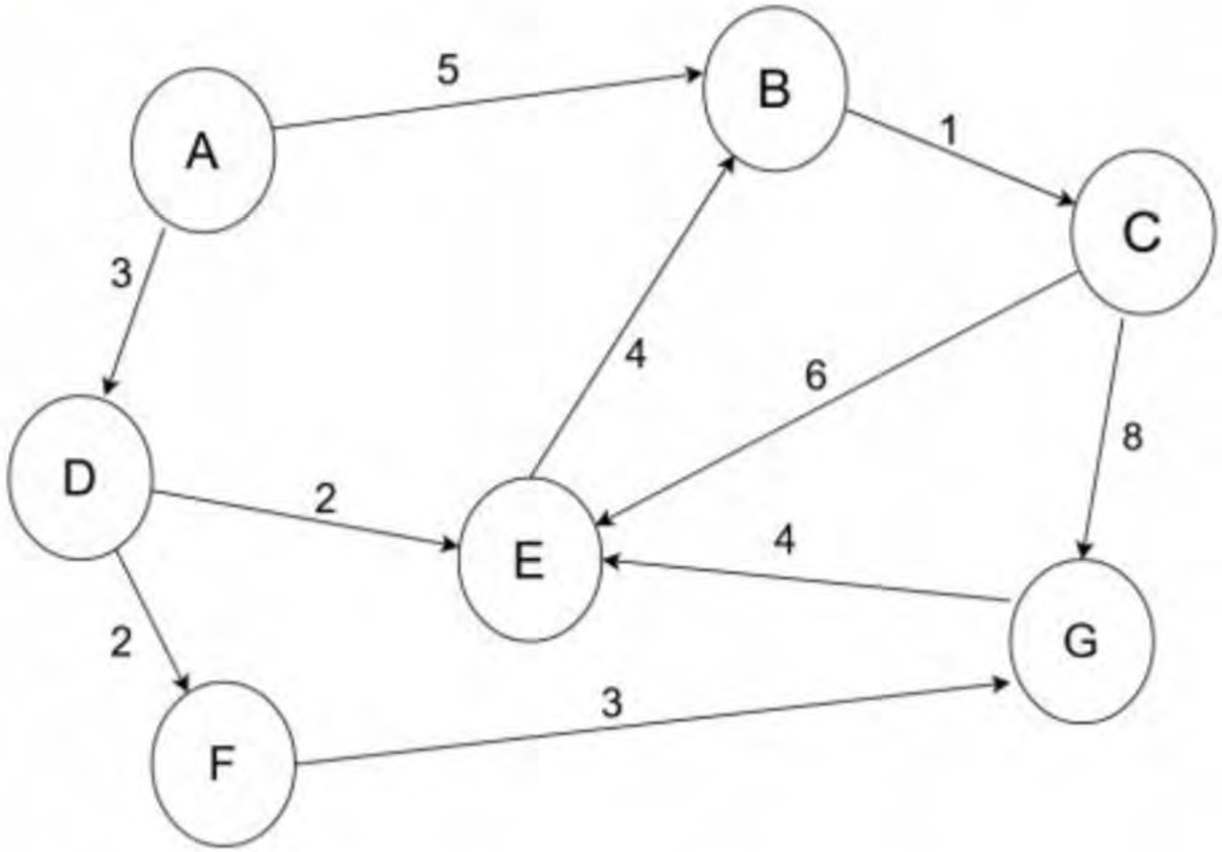
```
    4          2
(A)---(B)---(C)
 | \   |      |
6|  1\ |3    5|
 |    (D)---(E)
 |    / |     /|
 | 2/  |4   / |1
 | /   | /   /
(F)----(G)--(H)
    3      8
```

```
      4        3
A ------ B ------- C
| \     |        / |
|  \    |       /  |
|   2   1      4   2
|    \  |     /    |
D ----- E ---- F -- G
|  \    |    /     |
|   \   |   /      |
|    3  |  3       |
|     \ | /        |
H ----- I --------- J
      1      6
```


S
A
B
C
D
E
F
G1
G2
G3
5
6
9
6
2
2
3
2
1
2
7
7
9
5
8
www.educba.com

Uniform Cost search

Consider the following graph. Let the starting node be A and the goal node be G.



Finding path A to G using uniform cost search

```
          (4)
     S -------- A
    / \        / \
(2)/   \(3)  (2)   \(6)
  /     \   /       \
B        C           D
 \       / \         /
 (10)\(5)   \(3)  (4)
     \ /       \   /
      E         F
       \       /
       (2)\  (5)
          \ /
           G
```

THANK - YOU